

## Models in DBMS

High level or Conceptual — ER model/Diagram

Representational or implementation — Relations/Tables

Low level or physical data models — SQL (Queries)

In relational model,

"NULL" is used for not applicable and unknown data.

## Relational constraints

- ↳ Domain constraints (attributes should be atomic).
- ↳ Key constraints (All attributes cannot be same in multiple rows).
- ↳ Entity integrity constraints (Primary key should not have null value).
- ↳ Referential integrity constraints (Use of foreign key).

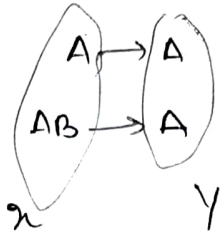
Super Key  $\subseteq$  Attributes  
 Candidate/Prime Key  $\subseteq$  Super Key.

Name is not a primary key  
 but even then cannot be NULL.  
 i.e. not NULL constraints.

- Referencing can be done within a single table also.
- A Super Key is generally a combination of all attributes.

## Functional Dependencies

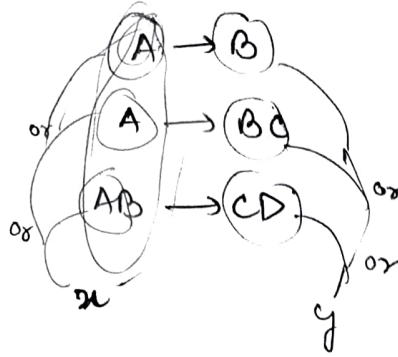
Trivial



$$Y \subseteq X$$

$$\begin{array}{l} A \subseteq A \\ A \subseteq AB \end{array}$$

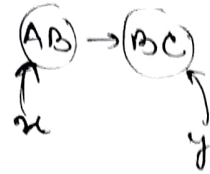
Non-trivial



$$X \cap Y = \emptyset$$

$$\begin{array}{l} A \cap B = \emptyset \\ A \cap BC = \emptyset \\ AB \cap CD = \emptyset \end{array}$$

Semi-non trivial



$$Y \not\subseteq X$$

$$X \cap Y \neq \emptyset$$

→ A decomposition is lossless if common attribute(s) after decomposition is/are prime attribute in any of the decomposed relation.

→ If a decomposition is lossy, cancel it, don't decompose.

### 1st Normal Form

→ No multivalued or composite attribute in the relation.

### 2nd Normal Form

→ No partial FD's, only full FD's.

### 3rd Normal Form

→ No Transitive Dependencies

### BCNF - Boyce Codd Normal Form

→ Determinants of all FDs must be Superkey.  
OR  
Determined attributes must be prime attribute.

# Relational Algebra

Operations -  $\pi$  : Projection       $\sigma$  : Selection       $\times$  : Cross-product  
                           $\cup$  : Union                       $-$  : Set Difference       $\rho$  : Rename

Expressions (sequence of operations)

|                                            |                                         |                                        |
|--------------------------------------------|-----------------------------------------|----------------------------------------|
| $\cap$ : Intersection<br>( $A - (A - B)$ ) | $\Join$ : Join<br>( $\nabla \times U$ ) | $/$ : Division<br>( $\pi, \times, -$ ) |
|--------------------------------------------|-----------------------------------------|----------------------------------------|

→ Selection : select the rows // Horizontal partitioning

→ Projection: selects the columns // Vertical partitioning.

→ It removes duplicates.

→ Rename : Useful when we create temporary table.

→ we can change the relation name or attribute name.

→ Union, Intersection & Set Diff.

compatibility - (i) Same degree  
(ii) Same Domain

→ Duplicates will be eliminated.

→ Cartesian Product : — Attributes get added  
Tuples get multiplied

→ Join — based on condition, if cond. is false or if tuples are NULL, no output.

|        | min          | max          |
|--------|--------------|--------------|
| U      | $\max(m, n)$ | $m+n$        |
| $\cap$ | 0            | $\min(m, n)$ |
| -      | 0            | m            |

|              | min          | max  |
|--------------|--------------|------|
| X            | mn           | mn   |
| <del>X</del> | 0            | mn   |
| <del>X</del> | m            | mn   |
| <del>X</del> | n            | mn   |
| <del>X</del> | $\max(m, n)$ | m, n |

## Relational Calculus

- Tuple Relational Calculus (TRC)
- Domain Relational Calculus (DRC)

### TRC

- one tuple variable range over all the tuples.
- one tuple for one relation.

### DRC

- one ~~domain~~ variables range over single attribute for all rows/tuples.
- for one relation, domain variables are = no. of attributes.

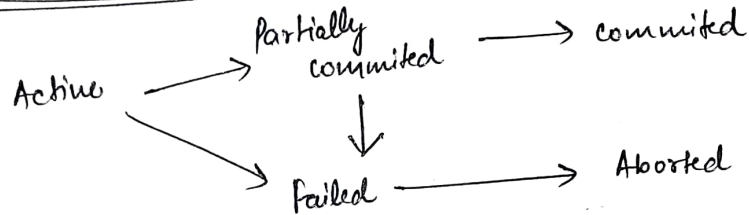
## SQL - Structured Query Language

- Schema - list of tables.
- SQL allows NULL values by default.
- No duplicate eliminated by default.  
Because it's costly, user wants it in aggregate.
- UNION will not give duplicate data by default.  
Use UNION ALL to get duplicate.
- In SQL every NULL value are different.
- NULL values are excluded for aggregate fun. operation.
- GROUP BY - ascending by default.
- Count (\*) should be group by all selected attributes.
- Every single null is group by itself.

## Transactions

- Txn - coll. of opr. that forms a single logic unit of work.
- ACID property
  - Atomicity → Consistency
  - Isolation → Durability.

## → Txn states



## → No. of Schedules

$T_1$  with  $n$  opr.  
 $T_2$  with  $m$  opr.

$$\begin{aligned} \text{Total schedule} \\ &= \frac{(n+m)!}{n!m!} \end{aligned}$$

$$\begin{aligned} \text{Total serial schedule} \\ &= 2! \end{aligned}$$

for  $n$  txn

$T_1 - a_1$   
 $T_2 - a_2$   
 $\vdots$   
 $T_n - a_n$

$$\text{Total schedule possible} = \frac{(a_1 + a_2 + a_3 + \dots + a_n)!}{(a_1! \cdot a_2! \cdot a_3! \cdot \dots \cdot a_n!)}$$

$$\text{Total serial schedule} = n!$$

$$\text{Total non-serial schedule} = \text{Total schedule} - \text{serial schedules.}$$

## → Types of Schedules

→ Serial schedule → No interleaving

→ Txns are consistent, always.

→ For  $n$  txns, total serial sch. possible =  $n!$

→ Complete schedule -

→ last ~~op~~ of each txn.  
either commit or abort.

→ Recoverable schedule -

Non-committed txns can be recovered when other txns roll-back. if read after written by first txn.  
A commit is done as per the rolling back txn.

→ Non-recoverable schedule -

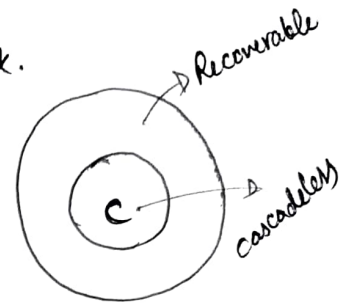
~~Schedule~~/Txn get lost even after committing due to roll back of another txn.

→ Cascading aborts (or) cascading roll-back schedule

→ failure of one txn cause other txn to roll back.

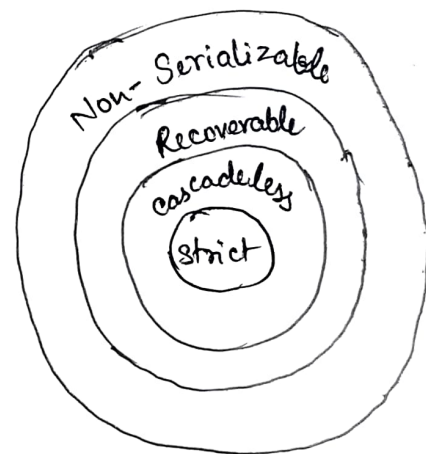
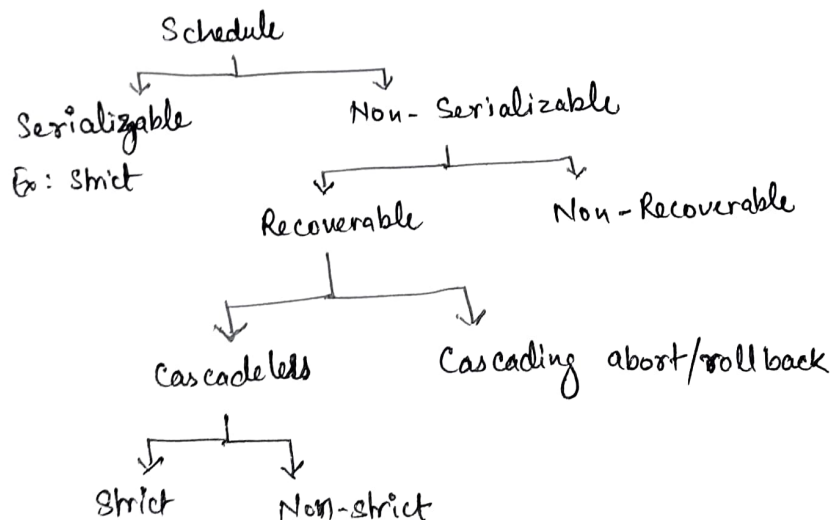
→ Cascadeless schedule

If  $T_i$  writes before  $T_j$  reads then,  $T_i$  should commit before  $T_j$  read it.



→ Strict schedule

If a ~~schedules~~<sup>txn</sup> writes then no other txn read or over-write the value until it either commit or abort.



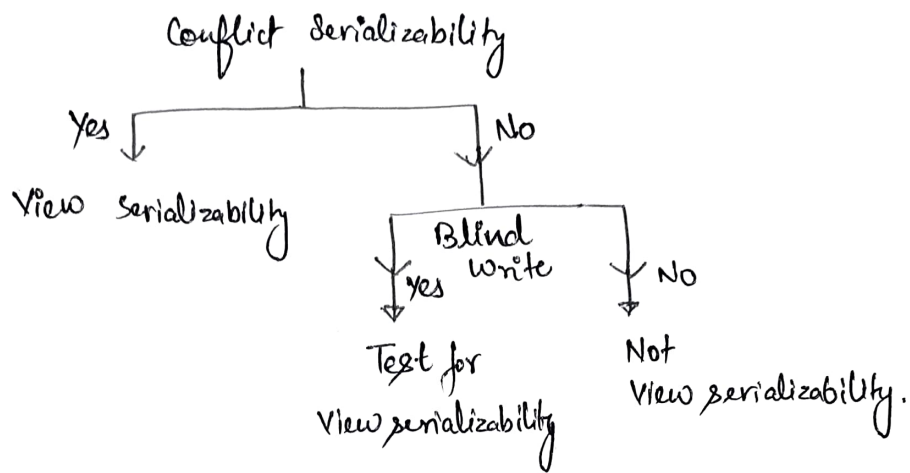
→ Serializability

→ Produce outcome equivalent to serial schedule.

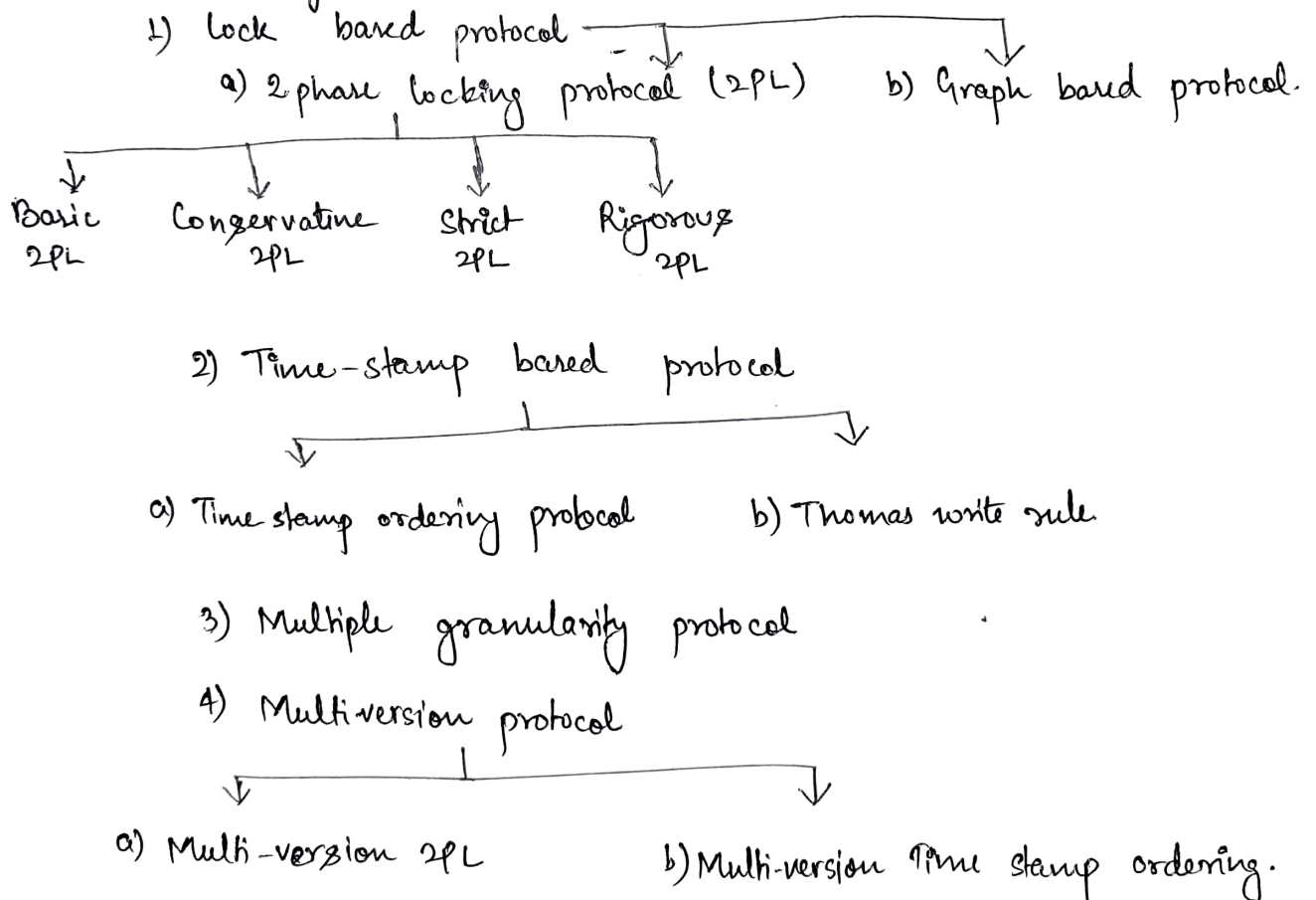
→ Result equivalent schedule.

→ Conflict equivalent schedule. / conflict serializability

→ View equivalent schedule. / View serializability.



### → Concurrency Control Protocol.

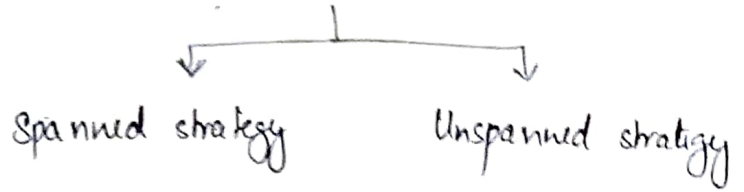




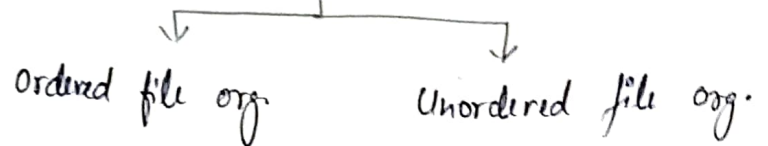
## File Structures



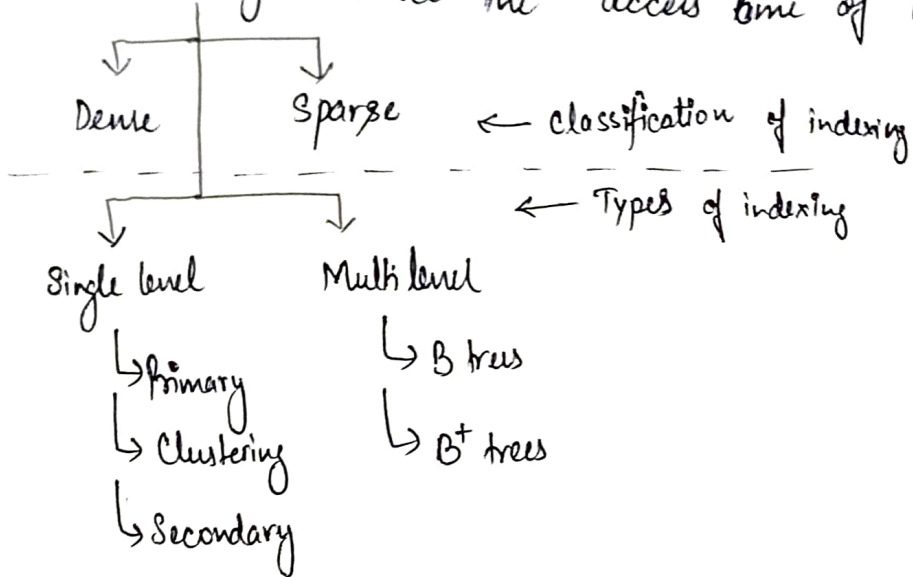
→ file storing strategies



→ Record organization



→ Indexing reduce the access time of block/data.



→ Primary indexing

→ ordered on primary key

→ Indexing over primary key

→ Index records = data blocks (count)

→ avg. access time for block

using indexing =  $\log_2(B_i) + 1$   
 $\uparrow$  no. of index block.

→ Clustering indexing

→ Ordered on non-key.

→ Non-key doesn't have distinct value.

→ block access time  $\geq \log_2(B_i) + 1$

→ Secondary indexing.

→ Unordered

→ Index records = Data records (count).



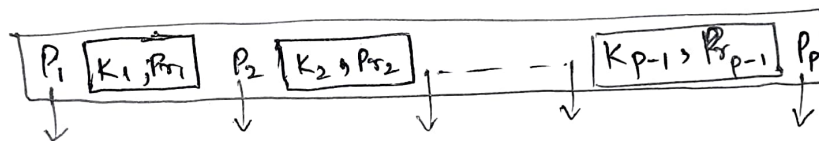
## B-Trees

|               | min (keys)                      | max (keys) | min (child)                 | max (children) |
|---------------|---------------------------------|------------|-----------------------------|----------------|
| Root          | 1                               | $p-1$      | 2                           | $p$            |
| Internal node | $\lceil \frac{p}{2} \rceil - 1$ | $p-1$      | $\lceil \frac{p}{2} \rceil$ | $p$            |
| leaf          | $\lceil \frac{p}{2} \rceil - 1$ | $p-1$      | —                           | —              |

$p \rightarrow$  order of tree  $\rightarrow$  max. no. of children a node can have.

$\rightarrow$  Internal node

$\langle P_1, \langle K_1, P_{r1} \rangle, P_2, \langle K_2, P_{r2} \rangle, \dots, \langle K_{p-1}, P_{rp-1} \rangle, P_p \rangle$



B-Tree Search

$\rightarrow$  Time complexity =  $O(\log_p n)$

$n =$  no. of search key

$p =$  order of B-Tree.

## B<sup>+</sup>Trees

|               | min child                   | max children | min key                         | max k |
|---------------|-----------------------------|--------------|---------------------------------|-------|
| Internal node | $\lceil \frac{p}{2} \rceil$ | $p$          | $\lceil \frac{p}{2} \rceil - 1$ | $p-1$ |

$\rightarrow$  Non-leaf node

$\langle P_1, \langle K_1 \rangle, P_2, \langle K_2 \rangle, \dots, \langle K_{p-1} \rangle, P_p \rangle$

$\rightarrow$  leaf node

$\langle \langle K_1, P_{r1} \rangle, \langle K_2, P_{r2} \rangle, \dots, \langle K_{q-1}, P_{rq-1} \rangle, P_{next} \rangle$

$\rightarrow$  Time complexity =  $O(\log_p n)$